

Arrays & strings

COMPLETE LEARNING GUIDE

What You'll Learn

Arrays and Strings are the two data structures every Java developer touches on day one, and getting their mechanics precisely right — fixed size, default values, reference semantics, and String's immutability — matters both for writing correct code and for the DSA modules ahead, which build directly on top of them. This guide covers how Java arrays are allocated and initialized (including multi-dimensional arrays, which are really arrays of arrays), the Arrays utility class, the core String API (substring, split, indexOf, and the ever-confusing `==` versus `.equals()`), and `StringBuilder` as the mutable alternative used whenever a string is built incrementally. Unlike the DSA "Arrays & strings" submodule, which focuses on algorithmic techniques applied to arrays and strings, this submodule focuses purely on Java language mechanics — the syntax and semantics you need before any algorithm can be written correctly.

1. Array Declaration, Fixed Size, and Default Values

A Java array is a fixed-size, homogeneous sequence of elements allocated on the heap; once created with `new int[10]`, its length can never change — "resizing" an array in Java always means allocating a brand-new array and copying elements into it, which is exactly what `ArrayList` does internally. Arrays are zero-indexed, and accessing an index outside `[0, length)` throws an `ArrayIndexOutOfBoundsException` at runtime rather than silently corrupting memory (unlike C). When an array is created without explicit values, every element is set to that type's default: 0 for numeric types, `false` for boolean, `'\u0000'` for char, and `null` for any reference/object type — reading an uninitialized reference-type element gives `null`, not a runtime error, until you try to call a method on it.

JAVA

```
int[] scores = new int[5];           // all elements default to 0
String[] names = new String[3];      // all elements default to null

int[] grades = {85, 90, 78};         // array literal, length fixed at 3
System.out.println(scores.length);   // 5 -- 'length' is a field, not a method

// Multi-dimensional array: really an array of arrays
int[][] grid = new int[3][4];        // 3 rows, each a length-4 int[]
grid[1][2] = 7;
```

2. Arrays Are Reference Types with Reference Semantics

Even an array of primitives is itself a reference type — the array variable holds a reference to a heap-allocated block, not the elements themselves. This means passing an array to a method passes a copy of the reference, not a copy of the data: modifications the method makes to array elements are visible to the caller after the method returns, unlike passing a primitive `int` (which is passed by value and cannot be modified by the callee). Assigning one array variable to another (`int[] b = a;`) makes both variables point at the identical array; mutating through `b` is visible through `a` as well, since there is only one underlying array object.

3. The Arrays Utility Class

`java.util.Arrays` provides static helper methods that operate on arrays without requiring manual loops for common operations: sorting, searching, filling, comparing, and converting to a readable string. `Arrays.toString()` is essential for debugging — printing an array directly with `System.out.println(arr)` prints its unhelpful hash-based reference string (`[I@1b6d3586`) rather than its contents, a mistake nearly every Java beginner makes at least once.

- `Arrays.sort(arr)` — sorts in place, ascending (Dual-Pivot Quicksort for primitives, $O(n \log n)$ average).
- `Arrays.binarySearch(arr, key)` — requires the array to already be sorted; returns the index or a negative insertion point.
- `Arrays.fill(arr, value)` — sets every element to the given value.
- `Arrays.equals(a, b)` — compares contents element-by-element (unlike `a == b`, which compares references).
- `Arrays.toString(arr)` / `Arrays.deepToString(arr)` — human-readable representation for 1D and nested arrays respectively.

4. String Immutability in Practice

Every `String` method that appears to modify a string — `substring()`, `replace()`, `toUpperCase()`, `trim()`, `concat()` — actually allocates and returns a new `String` object, leaving the original untouched. A classic beginner bug is calling `str.trim()`; and expecting `str` itself to change, when the correct usage is `str = str.trim()`;, reassigning the variable to the newly returned object. This immutability makes `String` inherently thread-safe and cacheable, but it means any code that builds up a string across many iterations — even something as simple as joining words in a loop — should use `StringBuilder` instead of `+=` to avoid allocating a new `String` object on every single iteration.

JAVA

```
StringBuilder sb = new StringBuilder();
for (String word : words) {
    sb.append(word).append(" "); // mutates the SAME buffer, no new object per iteration
}
String result = sb.toString().trim();
```

5. Core String Methods Every Interview Expects

A handful of String methods appear constantly in interview code and are worth knowing cold, including their exact boundary behavior: `substring(begin, end)` takes a start index inclusive and an end index exclusive, so `"hello".substring(1, 3)` returns `"el"`, not `"ell"`. `charAt(index)` throws `StringIndexOutOfBoundsException` past the string's length. `split(regex)` treats its argument as a regular expression, not a literal string, which surprises candidates who split on `"."` expecting a literal dot (it must be escaped as `"\\."` since a bare dot means "any character" in regex).

- `charAt(i)` — returns the char at index `i`; throws if out of bounds.
- `substring(begin, end)` — begin inclusive, end exclusive; `substring(begin)` goes to the end of the string.
- `indexOf(str)` — first occurrence index, or `-1` if not found.
- `split(regex)` — splits on a REGULAR EXPRESSION, not a literal; special regex characters (`.` `*` `+` `|`) must be escaped.
- `toArray()` — converts a String to a `char[]`, often the first step in string-manipulation algorithms.

6. Comparing Strings Correctly

As with boxed wrapper types, `==` on String compares references, and because of the string pool, two literals can appear equal by `==` while two runtime-constructed strings with identical content will not be — the only reliable way to compare String content is `.equals()` (case-sensitive) or `.equalsIgnoreCase()` (case-insensitive). For ordering comparisons (e.g. sorting), `.compareTo()` returns a negative, zero, or positive int based on lexicographic (dictionary, case-sensitive) order, following the same contract as `Comparable.compareTo()` used throughout the Collections framework.

Common Pitfalls

- Printing an array directly with `System.out.println(arr)` and getting an unreadable `[I@1b6d3586` instead of its contents — use `Arrays.toString(arr)`.
- Comparing Strings with `==` instead of `.equals()`, which works "by accident" for pooled literals but fails for strings built at runtime (e.g. from user input or concatenation).
- Calling `str.trim()` or `str.replace(...)` and expecting the original variable to change in place, forgetting Strings are immutable and the result must be reassigned.
- Using `str.split(".")` expecting a literal dot, when `split()` treats its argument as a regex and `"."` matches any character.
- Building a large string with `+=` inside a loop instead of `StringBuilder`, causing $O(n^2)$ time due to repeated full-string copies.
- Confusing `substring(begin, end)`'s exclusive end index, off-by-one errors are extremely common here.

- Assuming an array can be resized — attempting `arr.length = 10` or similar, when arrays are permanently fixed-size once created.

Key Takeaways

- Java arrays are fixed-size, zero-indexed, and default-initialized (`0/false/null`); "resizing" always means allocating a new array.
- Arrays are reference types — passing one to a method shares the same underlying data, unlike passing a primitive.
- Every String "mutation" method returns a new object — Strings themselves never change after creation.
- Use `StringBuilder` for repeated concatenation to avoid $O(n^2)$ behavior from repeated immutable-string copies.
- Always compare String content with `.equals()`, never `==` — pooling makes `==` unreliable for anything but literals.
- `substring(begin, end)` is begin-inclusive, end-exclusive — a frequent off-by-one trap.

Learn More

Oracle Java Tutorials — Arrays

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>

Oracle Java SE Docs — String (`java.lang.String`)

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

Oracle Java SE Docs — `StringBuilder`

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuilder.html>